Task 1: Launch an AWS EC2 Instance and Install Jenkins

Outcome

Jenkins was installed on an existing Amazon Linux 2023 EC2 instance using Java 21 (Amazon Corretto). The Jenkins web dashboard was successfully accessed through a browser on port 8080.

Objective

To install and configure Jenkins on an AWS EC2 instance and understand Java installation, SSH key-based authentication, package repository configuration, Linux service management, security group configuration, and browser-based access to Jenkins.

Prerequisites

- AWS account access.
- Permissions to access EC2 instances, key pairs, and security groups.
- An EC2 instance running Amazon Linux 2023.
- SSH client and a private key.
- Internet connectivity from the EC2 instance.
- Sufficient memory and disk space for a basic Jenkins lab.

Step-by-Step Implementation

Step 1: Generate an SSH Key Pair

Objective: Generate an SSH key pair for secure authentication to the EC2 instance.

Run the following command on the local machine where the key will be stored:

ssh-keygen -f linuxkey

This creates two files:

- linuxkey — private key used for SSH authentication.
- linuxkey.pub — public key imported into AWS.

Keep the private key secure. Never share it or include its contents in screenshots.

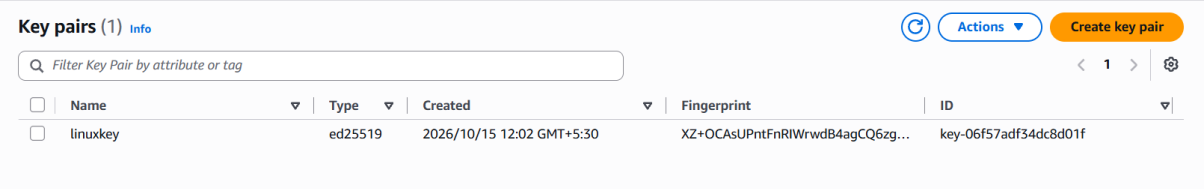
Step 2: Import the Public Key into AWS

Navigation path:

AWS Console → EC2 → Network & Security → Key Pairs → Import key pair

1. Enter the key pair name as linuxkey.
2. Upload the linuxkey.pub file or paste its public-key contents.
3. Click Import key pair.
4. Verify that linuxkey appears in the EC2 Key Pairs list.

Screenshot Evidence



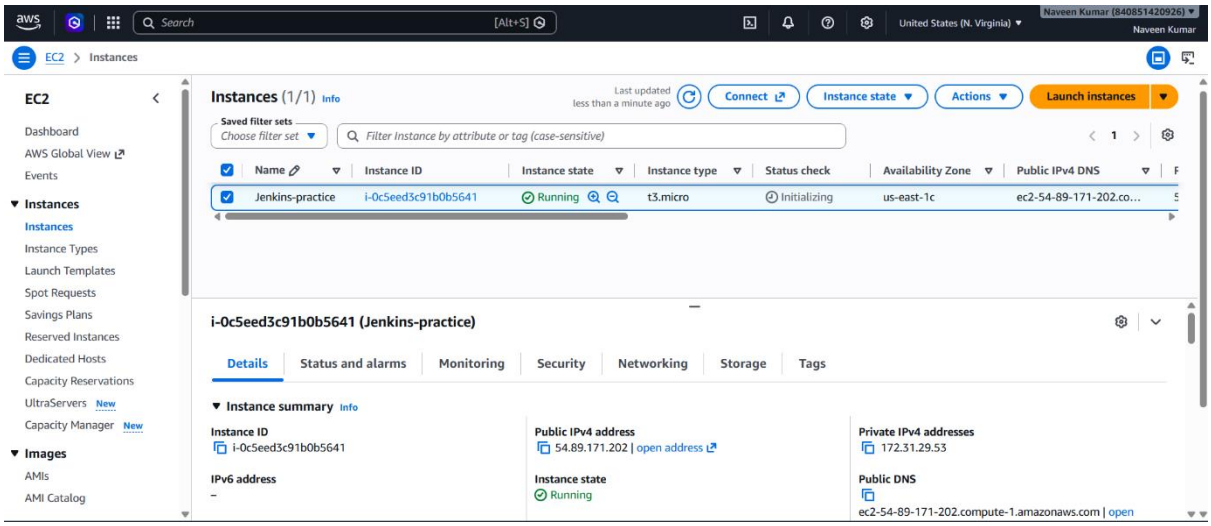
Step 3: Launch or Select the EC2 Instance

During instance launch, select the imported linuxkey key pair under Key pair (login).

Parameter	Value
Cloud provider	AWS
Service	Amazon EC2
Operating system	Amazon Linux 2023
Login user	ec2-user
Key pair	Linuxkey

For this assignment, Jenkins was installed on an existing Amazon Linux 2023 instance. If documenting a newly launched instance, capture the key pair selection during launch.

Screenshot Evidence



Step 4: Connect to the EC2 Instance

Connect using an SSH client and the private key.

ssh -i "linuxkey" ec2-user@YOUR_EC2_PUBLIC_IP

Replace YOUR_EC2_PUBLIC_IP with the instance's public IPv4 address.

Verify the current user:

whoami

Expected output:

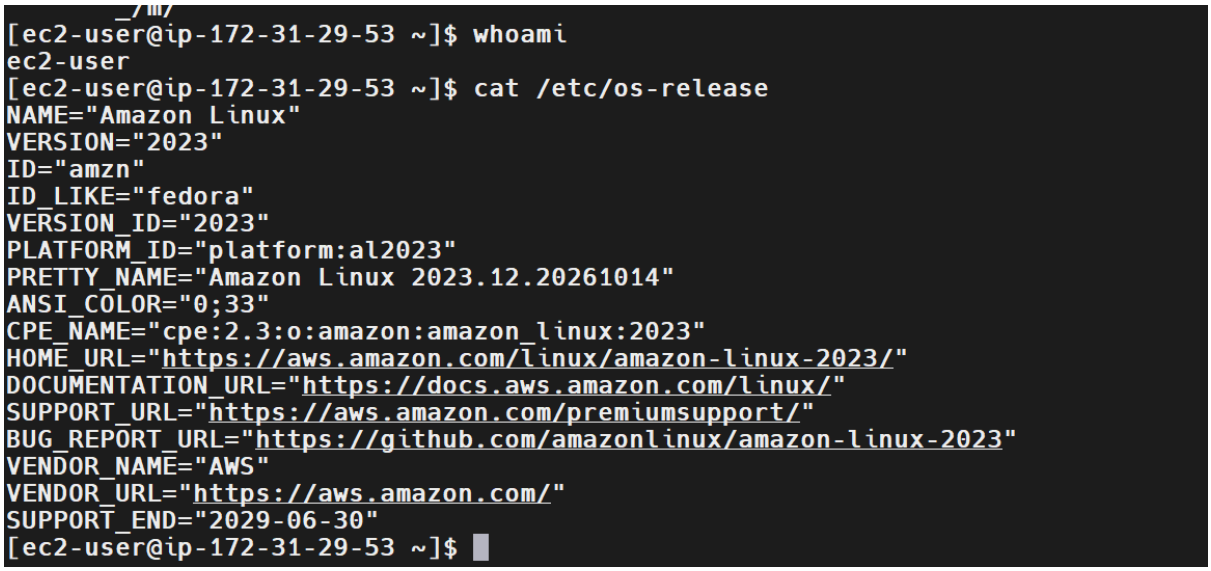
ec2-user

Check the operating system:

cat /etc/os-release

The actual output confirmed Amazon Linux 2023.

Screenshot Evidence



Step 5: Check System Resources

Run the following commands:

free -h

df -h

The observed resource values were:

Resource	Observed value
Total memory	913 MiB
Available memory	617 MiB
Root filesystem size	8 GB
Available root disk space	6.4 GB
Root filesystem usage	20%

The instance had limited memory, so it was suitable for a basic installation exercise but might struggle with larger Jenkins builds.

Step 6: Install Java 21

Jenkins requires a compatible Java runtime. Java 21 (Amazon Corretto) was installed using the Amazon Linux package manager.

Run:

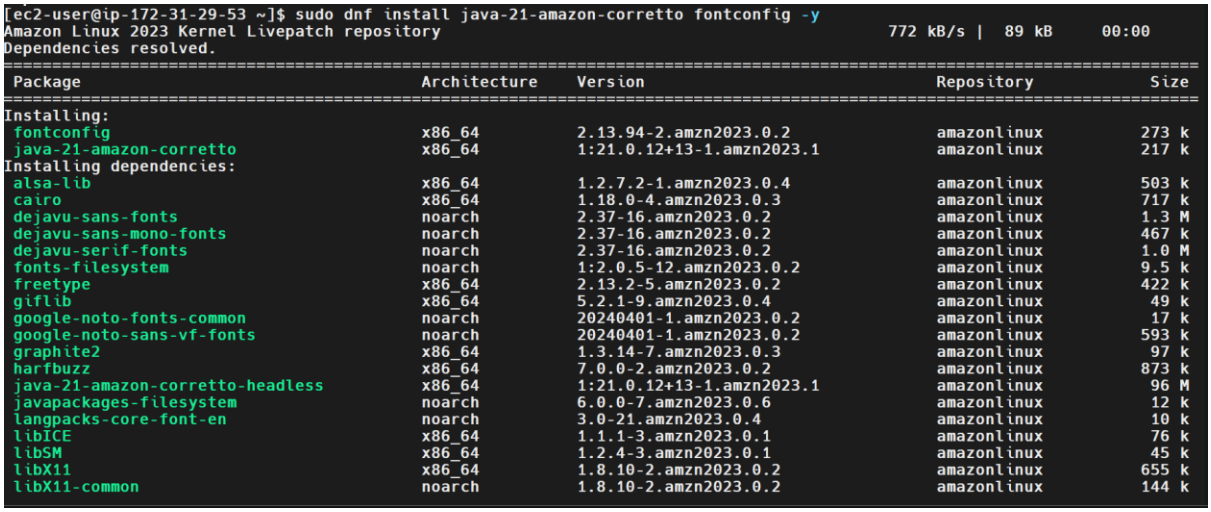
sudo dnf install java-21-amazon-corretto fontconfig -y

Verify the installation:

java --version

The actual output confirmed OpenJDK 21 running Amazon Corretto 21.0.12.1 LTS.

Screenshot Evidence



```
[ec2-user@ip-172-31-29-53 ~]$ sudo dnf install java-21-amazon-corretto fontconfig -y
Amazon Linux 2023 Kernel Livepatch repository                    772 kB/s |  89 kB     00:00
Dependencies resolved.
================================================================================================
 Package                          Architecture    Version                          Repository      Size
================================================================================================
Installing:
 fontconfig                       x86_64          2.13.94-2.amzn2023.0.2           amazonlinux    273 k
 java-21-amazon-corretto          x86_64          1:21.0.12+13-1.amzn2023.1        amazonlinux    217 k
Installing dependencies:
 alsa-lib                         x86_64          1.2.7.2-1.amzn2023.0.4           amazonlinux    503 k
 cairo                            x86_64          1.18.0-4.amzn2023.0.3            amazonlinux    717 k
 dejavu-sans-fonts                noarch          2.37-16.amzn2023.0.2             amazonlinux    1.3 M
 dejavu-sans-mono-fonts           noarch          2.37-16.amzn2023.0.2             amazonlinux    467 k
 dejavu-serif-fonts               noarch          2.37-16.amzn2023.0.2             amazonlinux    1.0 M
 fonts-filesystem                 noarch          1:2.0.5-12.amzn2023.0.2          amazonlinux    9.5 k
 freetype                         x86_64          2.13.2-5.amzn2023.0.2            amazonlinux    422 k
 giflib                           x86_64          5.2.1-9.amzn2023.0.4             amazonlinux     49 k
 google-noto-fonts-common         noarch          20240401-1.amzn2023.0.2          amazonlinux     17 k
 google-noto-sans-vf-fonts        noarch          20240401-1.amzn2023.0.2          amazonlinux    593 k
 graphite2                        x86_64          1.3.14-7.amzn2023.0.3            amazonlinux     97 k
 harfbuzz                         x86_64          7.0.0-2.amzn2023.0.2             amazonlinux    873 k
 java-21-amazon-corretto-headless x86_64          1:21.0.12+13-1.amzn2023.1        amazonlinux     96 M
 javapackages-filesystem          noarch          6.0.0-7.amzn2023.0.6             amazonlinux     12 k
 langpacks-core-font-en           noarch          3.0-21.amzn2023.0.4              amazonlinux     10 k
 libICE                           x86_64          1.1.1-3.amzn2023.0.1             amazonlinux     76 k
 libSM                            x86_64          1.2.4-3.amzn2023.0.1             amazonlinux     45 k
 libX11                           x86_64          1.8.10-2.amzn2023.0.2            amazonlinux    655 k
 libX11-common                    noarch          1.8.10-2.amzn2023.0.2            amazonlinux    144 k
```

Step 7: Configure the Jenkins Repository

Download the Jenkins stable RPM repository configuration:

sudo wget -O /etc/yum.repos.d/jenkins.repo \

https://pkg.jenkins.io/rpm-stable/jenkins.repo

Import the Jenkins signing key using the key URL recommended by the official Jenkins installation instructions:

sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key

This configures the package source and imports the signing key used to verify Jenkins packages.

Step 8: Install Jenkins

Install Jenkins using the Amazon Linux package manager:

sudo dnf install jenkins -y

Wait for the installation to complete and review the output for errors.

Step 9: Start and Enable Jenkins

Reload systemd configuration and start Jenkins:

sudo systemctl daemon-reload

sudo systemctl enable --now jenkins

Check the service status:

sudo systemctl status jenkins --no-pager

A successful service check should show:

Active: active (running)

Check whether Jenkins is listening on port 8080:

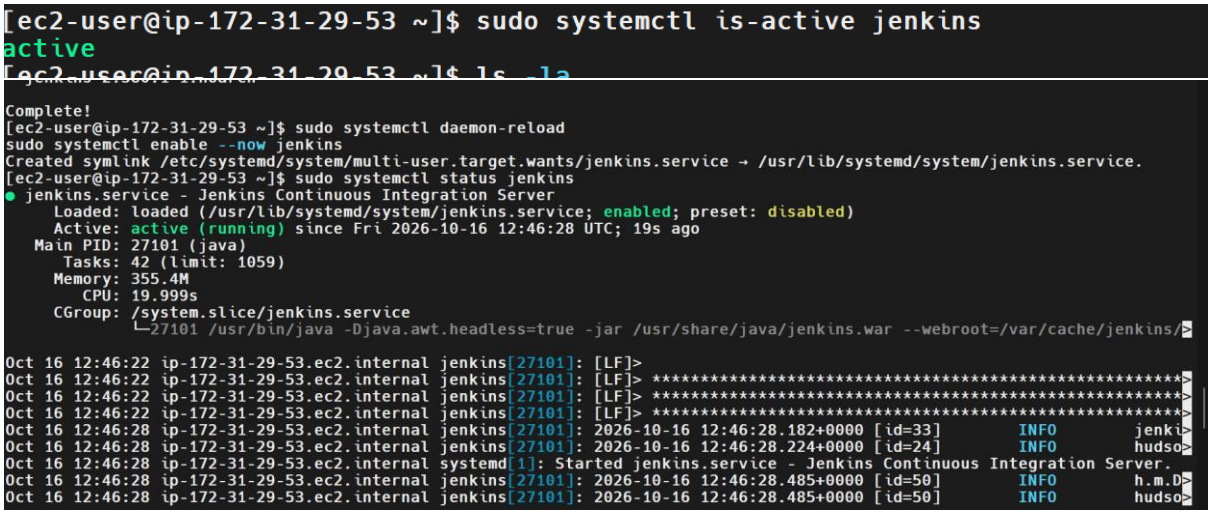
sudo ss -lntp | grep 8080

The actual output provided was:

LISTEN 0 50 *:8080 *:* users:(("java",pid=27101,fd=9))

This confirmed that a Java process was listening on port 8080.

Screenshot Evidence



Step 10: Troubleshoot Jenkins Browser Connectivity

Objective: Make the Jenkins web interface accessible through the EC2 instance's public IPv4 address.

Initially, accessing Jenkins through the public IP address and port 8080 did not work. The EC2 security group inbound rules were reviewed and modified to permit Jenkins web traffic. After the rule was updated, the browser connection succeeded.

Navigation path:

AWS Console → EC2 → Instances → Select instance

→ Security → Security groups → Edit inbound rules

Add or verify the following rule:

Parameter	Recommended value
Type	Custom TCP
Protocol	TCP

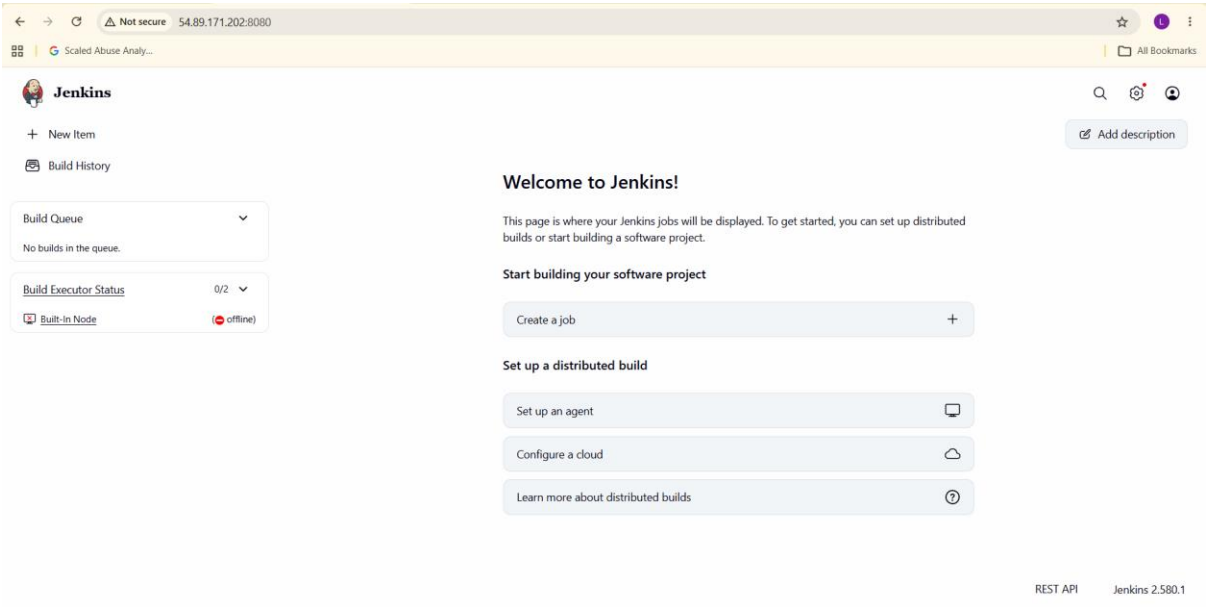
Port range 8080

Source Your public IP address with /32

During troubleshooting, all traffic was temporarily allowed, according to the task notes. This is an overly permissive configuration and should not be left in place. Replace it with a rule that allows only TCP port 8080 from a trusted source.

SSH port 22 should also be restricted to your own IP address wherever possible.

Screenshot Evidence



Step 11: Access the Jenkins Dashboard

Open a web browser and navigate to:

http://YOUR_EC2_PUBLIC_IP:8080

Replace the placeholder with the actual public IPv4 address of the EC2 instance.

The supplied screenshot showed the Jenkins welcome dashboard, including options to create a job and configure build agents. The footer displayed Jenkins version 2.580.1.

This confirmed that the Jenkins web interface was accessible.

Validation Steps

Validation	Command or check	Result
Operating system	cat /etc/os-release	Amazon Linux 2023 confirmed
Java runtime	java –version	Java 21 confirmed
Jenkins port	sudo ss -lntp | grep 8080	Java process listening on port 8080 confirmed
Browser access	Open public IP on port 8080	Jenkins dashboard accessed successfully

Service status	sudo systemctl is-active Jenkins	Capture the actual output to complete verification
Security group	Check inbound rules	Confirm port 8080 is restricted to a trusted source

Explanation

Jenkins is an automation server used to implement continuous integration and continuous delivery (CI/CD) pipelines. It can automate source-code checkout, building, testing, packaging, and deployment. Hosting Jenkins on EC2 provides a cloud-based environment for learning and managing software delivery workflows.

Java is required because Jenkins runs on the Java Virtual Machine. The Jenkins package repository enables installation through the Linux package manager, while systemd manages the Jenkins service. Port 8080 is the default HTTP port for accessing the Jenkins web interface. SSH key pairs provide secure authentication to the EC2 instance, while security group rules control inbound network access.

Real-Time Use Case

In a software development organization, developers push code to a Git repository. Jenkins can trigger a pipeline that checks out the code, builds the application, runs tests, and generates deployment artifacts. The pipeline can then deploy the application to a test environment, reducing manual effort and improving delivery consistency.

Common Errors and Troubleshooting

Error or issue	**Possible cause**	**Resolution**
SSH connection fails	Incorrect username, key, or security group rule	Verify the OS username, private key, public IP, and SSH inbound rule
Java command not found	Java is not installed or not on PATH	Install a compatible Java runtime and verify with java --version
Jenkins package not found	Repository configuration is missing or invalid	Verify the repository file and package metadata
Jenkins service fails to start	Memory pressure, Java mismatch, or configuration issue	Check sudo journalctl -u jenkins -n 50 --no-pager
Browser cannot connect to port 8080	Security group, routing, public IP, or service issue	Verify service status, port listener, public IP, and inbound rules
Disk space becomes low	Logs or build artifacts consume storage	Check df -h and du -sh before removing files
Built-In Node shows offline	Agent configuration or resource issue	Inspect node status and Jenkins logs; the web service may still be available

Conclusion:

Jenkins was installed on Amazon Linux 2023 with Java 21. A Java process was confirmed listening on port 8080, and the Jenkins dashboard was successfully accessed through the browser after updating the EC2 security group.

Task 2: Install and Configure Apache Tomcat on an Ubuntu EC2 Instance

Date: 2026-10-16

1. Outcome

Successfully launched an Ubuntu EC2 instance on AWS, installed Java 21 and Apache Tomcat, verified that Tomcat was listening on port 8080, and accessed the Tomcat web page through a browser using the instance's public IPv4 address.

2. Objective

The objective of this task was to deploy Apache Tomcat on an Ubuntu Linux server hosted on AWS EC2. This task demonstrates the installation of Java, application server configuration, Linux service management, port verification, AWS security group configuration, and external web access validation.

3. Prerequisites

- AWS account with permission to launch and manage EC2 instances.
- Ubuntu Server EC2 instance.
- SSH client such as MobaXterm.
- SSH key pair for secure access.
- Internet connectivity for package installation.
- Security group configured to permit SSH access from a trusted IP.
- Security group rule for Tomcat access on TCP port 8080.

4. Step-by-Step Implementation

Step 1: Launch an Ubuntu EC2 Instance

Procedure:

1. Logged in to the AWS Management Console.
2. Navigated to EC2 → Instances → Launch instances.
3. Configured the instance with the following settings.

Parameter	Configuration
Instance name	tomcat-ubuntu-lab
Operating system	Ubuntu Server
Version	Ubuntu 26.04, as confirmed by the installed package version

Instance type	As selected during launch
Key pair	linuxkey
Storage	As configured during launch
Network	AWS VPC and associated subnet
SSH access	TCP port 22, restricted to the trusted source IP

4. Launched the instance and waited until the instance state showed Running.
5. Verified that the instance passed its AWS status checks.

Screenshot 1: Capture the EC2 instance details showing the instance name, state, public IPv4 address, and key pair name.

Step 2: Connect to Ubuntu Using MobaXterm

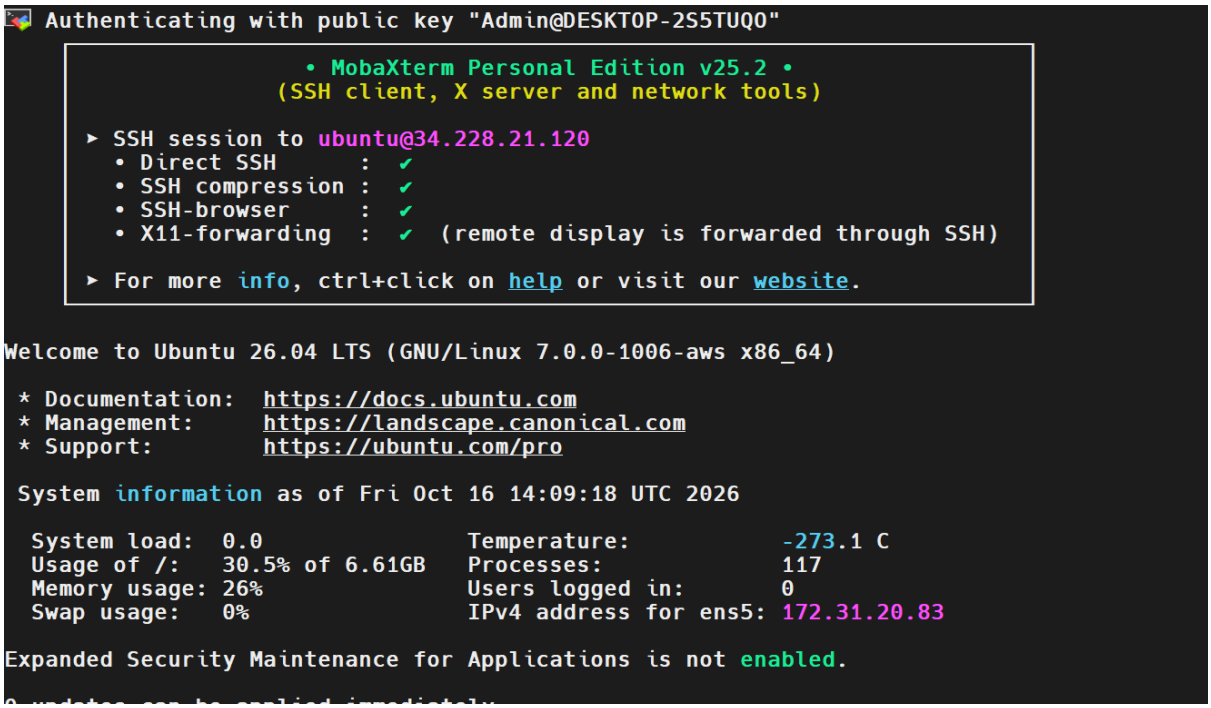
Procedure:

1. Opened MobaXterm.
2. Created an SSH session using the public IPv4 address of the Ubuntu EC2 instance.
3. Configured the SSH username as ubuntu.
4. Selected the private key corresponding to the linuxkey key pair.
5. Connected successfully to the instance.

The terminal displayed the following prompt:

ubuntu@ip-172-31-20-83:~$

This confirmed successful SSH access to the Ubuntu instance.



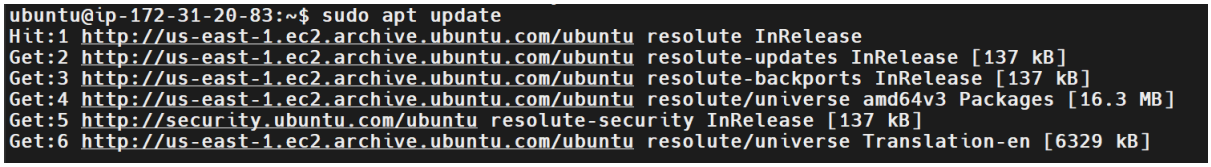
Step 3: Update Ubuntu Package Information

Executed the following command to refresh the package repository information:

sudo apt update

This command retrieves the latest package metadata from the configured Ubuntu repositories so that packages can be installed using current repository information.

Screenshot 3:



Step 4: Install Java 21

Apache Tomcat requires Java to run because it is a Java-based application server.

Installed the Java Development Kit using:

sudo apt install openjdk-21-jdk -y

Explanation:

- sudo executes the command with administrative privileges.
- apt install installs a package from the configured Ubuntu repositories.
- openjdk-21-jdk installs Java 21 and its development tools.
- -y automatically confirms the package installation prompts.

Verified the Java installation:

java -version

Actual verification output:

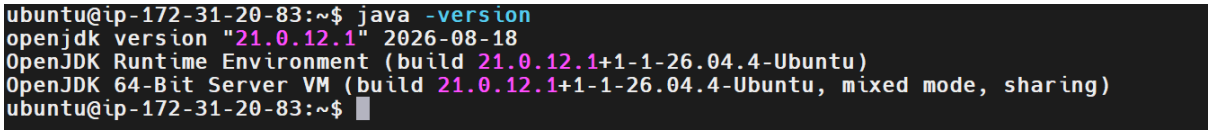
openjdk version "21.0.12.1" 2026-08-18

OpenJDK Runtime Environment (build 21.0.12.1+1-1-26.04.4-Ubuntu)

OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-26.04.4-Ubuntu, mixed mode, sharing)

This confirmed that Java 21 was installed successfully.

Screenshot 4:



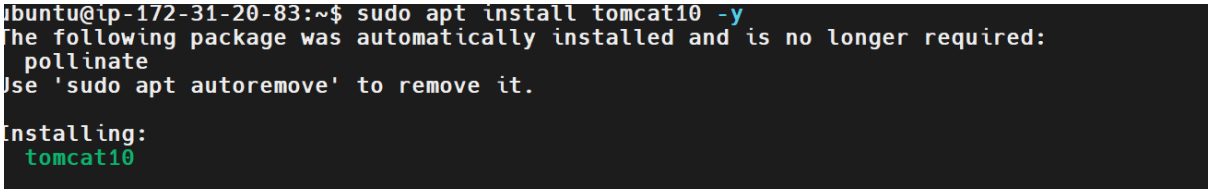
Step 5: Install Apache Tomcat

Installed Apache Tomcat 10 using the Ubuntu package manager:

sudo apt install tomcat10 -y

This command installs the Tomcat 10 application server and its required dependencies from the configured package repositories.

Screenshot 5



Step 6: Start and Enable the Tomcat Service

Executed:

sudo systemctl enable --now tomcat10

Explanation:

- systemctl manages system services.
- enable configures the service to start automatically at boot.
- --now starts the service immediately.

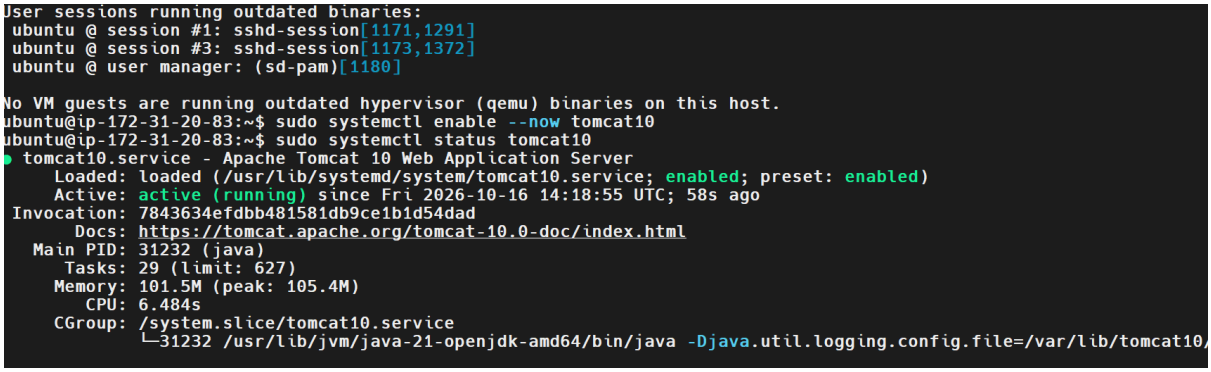
Verified the service status:

sudo systemctl status tomcat10

The expected successful state is:

Active: active (running)

Screenshot 6:



Step 7: Verify Tomcat Is Listening on Port 8080

Executed:

sudo ss -lntp | grep 8080

Actual output:

LISTEN 0 100 *:8080 *:* users:(("java",pid=31232,fd=40))

Explanation:

- ss displays socket information.

- -l displays listening sockets.
- -n displays numeric addresses and port numbers.
- -t displays TCP sockets.
- -p displays process information.
- grep 8080 filters the output for port 8080.

The output confirmed that a Java process was listening on port 8080, the port used by the Tomcat service in this setup.

Step 8: Test Tomcat Locally

Executed:

curl -I http://localhost:8080

Actual output:

HTTP/1.1 200

Accept-Ranges: bytes

ETag: W/"1905-1792160335176"

Last-Modified: Fri, 16 Oct 2026 14:18:55 GMT

Content-Type: text/html

Content-Length: 1905

Date: Fri, 16 Oct 2026 14:21:50 GMT

Explanation:

The curl -I command sends an HTTP request and displays the response headers. The HTTP/1.1 200 response confirms that the server successfully handled the local HTTP request.

Screenshot 8:

```
ubuntu@ip-172-31-20-83:~$ sudo ss -lntp | grep 8080
LISTEN 0      100                *:8080            *:*    users:(("java",pid=31232,fd=40))
ubuntu@ip-172-31-20-83:~$ curl -I http://localhost:8080
HTTP/1.1 200
Accept-Ranges: bytes
ETag: W/"1905-1792160335176"
Last-Modified: Fri, 16 Oct 2026 14:18:55 GMT
Content-Type: text/html
Content-Length: 1905
Date: Fri, 16 Oct 2026 14:21:50 GMT
```

Step 9: Configure the AWS Security Group

To access Tomcat from a browser outside the EC2 instance, configured the attached EC2 security group.

Procedure:

1. Opened AWS Console → EC2 → Instances.
2. Selected the Ubuntu EC2 instance.

3. Opened the Security tab.
4. Selected the attached security group.
5. Chose Edit inbound rules.
6. Added the following rule.

Setting	Value
Type	All traffic
Protocol	all
Port range	all
Source	custom

7. Saved the inbound rule.

Step 10: Access Tomcat Through a Web Browser

Opened the following URL in a web browser:

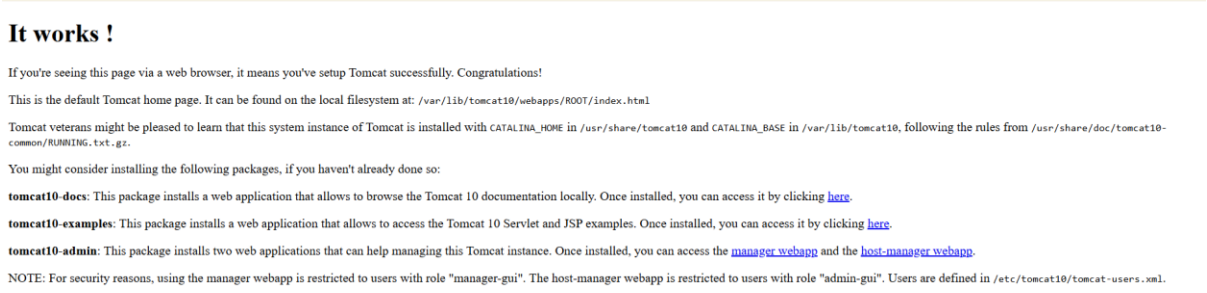
http://YOUR_EC2_PUBLIC_IP:8080

Replaced YOUR_EC2_PUBLIC_IP with the actual public IPv4 address of the Ubuntu EC2 instance.

The Apache Tomcat web page was displayed successfully in the browser.

This confirmed that Tomcat was reachable externally through the EC2 public IP and the configured security group.

Screenshot 10:



5. Validation Steps

Validation	Command or method	Expected result	Status
Ubuntu SSH access	Connect through MobaXterm	Ubuntu shell prompt	Passed
Java installation	java -version	Java 21 version details	Passed
Tomcat installation	sudo apt install tomcat10 -y	Package installation completes	Completed

Tomcat port	sudo ss -lntp | grep 8080	Java process listening on port 8080	Passed
Local HTTP request	curl -I http://localhost:8080	HTTP 200 response	Passed
External browser access	Public IP with port 8080	Tomcat web page loads	Passed

6. Explanation

Apache Tomcat is an open-source Java application server and servlet container used to run Java web applications. In this task, Tomcat 10 was installed on an Ubuntu EC2 instance, with OpenJDK 21 providing the Java runtime. The Tomcat service was configured to start automatically and verified to be listening on TCP port 8080.

The systemctl command was used to manage the Tomcat service, while ss and curl helped validate network listening and HTTP responses. Finally, an AWS security group rule allowed access to port 8080 from a trusted IP address. Successfully opening the Tomcat page in the browser demonstrated that the application server was reachable through the EC2 public IPv4 address.

7. Real-Time Use Case

Apache Tomcat is commonly used to host Java web applications in development, testing, and production environments. For example, an organization may deploy a Java-based application to Tomcat on an EC2 instance so that users or other application components can access it over HTTP.

In a DevOps environment, Tomcat deployments can be integrated into CI/CD pipelines using tools such as Jenkins. A pipeline can build a Java application, run automated tests, package it as a WAR file, and deploy it to Tomcat. Monitoring, restricted network access, automated backups, and controlled release procedures help improve application reliability and security.

8. Common Errors and Troubleshooting

Error or issue	**Possible cause**	**Solution**
Server refused our key	Incorrect private key or SSH username	Verify the matching key pair and use the ubuntu username.
java: command not found	Java is not installed or is not in the PATH	Install OpenJDK and verify using java -version.
Tomcat service fails to start	Package, Java, or configuration issue	Check sudo systemctl status tomcat10 and sudo journalctl -u tomcat10 -n 50 --no-pager.
Port 8080 is not listening	Tomcat is stopped or failed to start	Run sudo systemctl restart tomcat10 and check the service logs.
Local curl fails	Tomcat is not responding locally	Check service status, logs, and listening ports.

Browser cannot access Tomcat	Security group, public IP, routing, or host firewall issue	Verify the instance's public IPv4 address, inbound TCP 8080 rule, and network connectivity.
Browser displays an unexpected page	Different application or default application configuration	Check the URL, Tomcat deployment, and service configuration.

9. Security and Cost Considerations

- Restrict SSH access to trusted IP addresses.
- Restrict TCP port 8080 to the intended source IP addresses.
- Avoid leaving temporary inbound rules open to the entire internet.
- Stop or terminate the EC2 instance when the lab is complete if it is no longer required.
- Review attached EBS volumes and their deletion settings before terminating the instance.
- Remember that stopping or terminating an instance does not necessarily delete its EBS volumes, and some AWS resources may continue to incur charges.

10. Conclusion

Task 2 completed successfully.

Apache Tomcat 10 was installed on an Ubuntu EC2 instance with Java 21. The Tomcat process was confirmed listening on port 8080, the local HTTP request returned HTTP/1.1 200, and the Tomcat web page was successfully accessed through the EC2 public IPv4 address using a browser.

Task 3: Install and Configure Docker Desktop on Windows 11

Date: 2026-10-16

1. Outcome

Successfully installed and started Docker Desktop on Windows 11, configured Windows Subsystem for Linux (WSL 2), verified Docker CLI availability, and executed the hello-world container successfully. The container exited with status code 0, confirming successful execution.

2. Objective

The objective of this task was to install Docker Desktop on Windows 11 and verify the Docker environment. This task demonstrates WSL 2 setup, Docker Desktop configuration, Docker CLI commands, image execution, container lifecycle management, and basic container verification.

3. Prerequisites

- Windows 11 computer.
- Administrator access to install Windows components.
- Hardware virtualization enabled.
- Windows Subsystem for Linux (WSL 2).

- Internet connectivity for downloading Docker Desktop and container images.
- Sufficient system memory and disk space.

4. Step-by-Step Implementation

Step 1: Verify Windows Version and Virtualization

Checked the Windows version using:

winver

The system reported the following Windows version:

10.0.26200.9711

Hardware virtualization was checked using Task Manager under Performance → CPU.

Step 2: Install Windows Subsystem for Linux

Opened PowerShell with administrator privileges and executed:

wsl --install

When WSL was not installed, Windows displayed an installation message. The required WSL components were enabled with:

wsl --install --no-distribution

The command returned:

The requested operation is successful. Changes will not be effective until the system is rebooted.

Restarted Windows to apply the changes.

Screenshot 2:

```
PS C:\Users\Admin> wsl --install
Downloading: Windows Subsystem for Linux 3.0.3
Installing: Windows Subsystem for Linux 3.0.3
Windows Subsystem for Linux 3.0.3 has been installed.
Installing Windows optional component: VirtualMachinePlatform

Deployment Image Servicing and Management tool
Version: 10.0.26100.9549

Image Version: 10.0.26200.9711

Enabling feature(s)
[==========================100.0%==========================]
The operation completed successfully.
The requested operation is successful. Changes will not be effective until the system is rebooted.
The requested operation is successful. Changes will not be effective until the system is rebooted.
PS C:\Users\Admin> wsl --update
Checking for updates.
The most recent version of Windows Subsystem for Linux is already installed.
PS C:\Users\Admin> wsl --status
Default Version: 2
WSL1 is not supported with your current machine configuration.
Please enable the "Windows Subsystem for Linux" optional component to use WSL1.
WSL2 is unable to start since virtualization is not enabled on this machine.
Please ensure the "Virtual Machine Platform" optional component is enabled and virtualization is turned on in your compu
ter's firmware settings.
```

Step 3: Install and Configure Ubuntu on WSL

Installed Ubuntu using:

wsl --install Ubuntu

Completed the initial Ubuntu setup by creating a default Unix user account named ubuntu and setting its password.

The terminal displayed:

Create a default Unix user account: ubuntu

passwd: password updated successfully

Verified the WSL configuration from PowerShell:

wsl --status

The system reported:

Default Version: 2

Also checked installed Linux distributions with:

wsl --list --verbose

Step 4: Install Docker Desktop

Downloaded Docker Desktop from the official Docker documentation:

https://docs.docker.com/desktop/setup/install/windows-install/

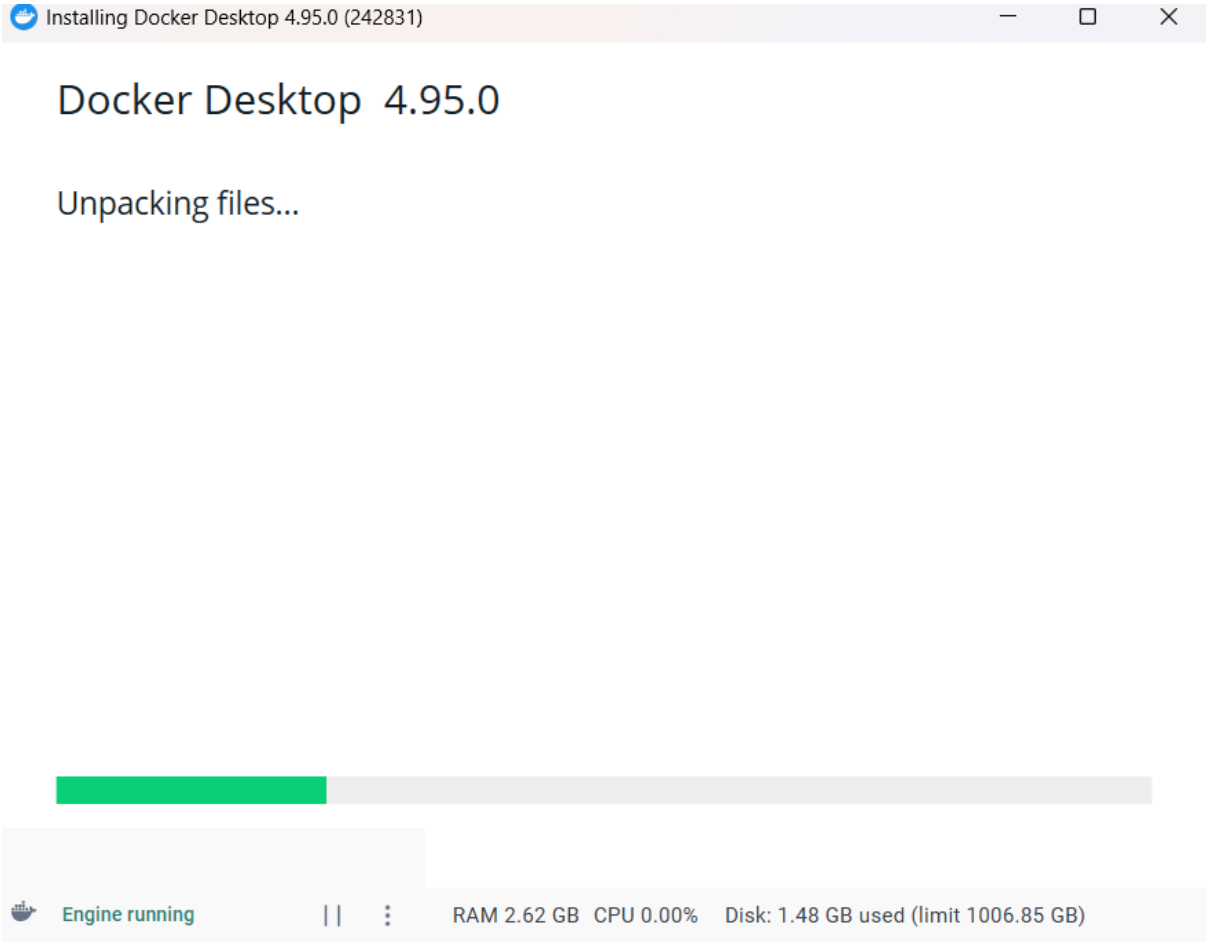
Completed the installation and launched Docker Desktop. Skipped account sign-in during the initial welcome screen and waited for the Docker engine to start.

The Docker Desktop dashboard showed:

Engine running

This confirmed that Docker Desktop had started successfully.

Screenshot 4:



Step 5: Verify Docker CLI

Opened a new PowerShell window and executed:

docker --version

This command verifies that the Docker CLI is available from the Windows command line.

Expected output: Docker version information.

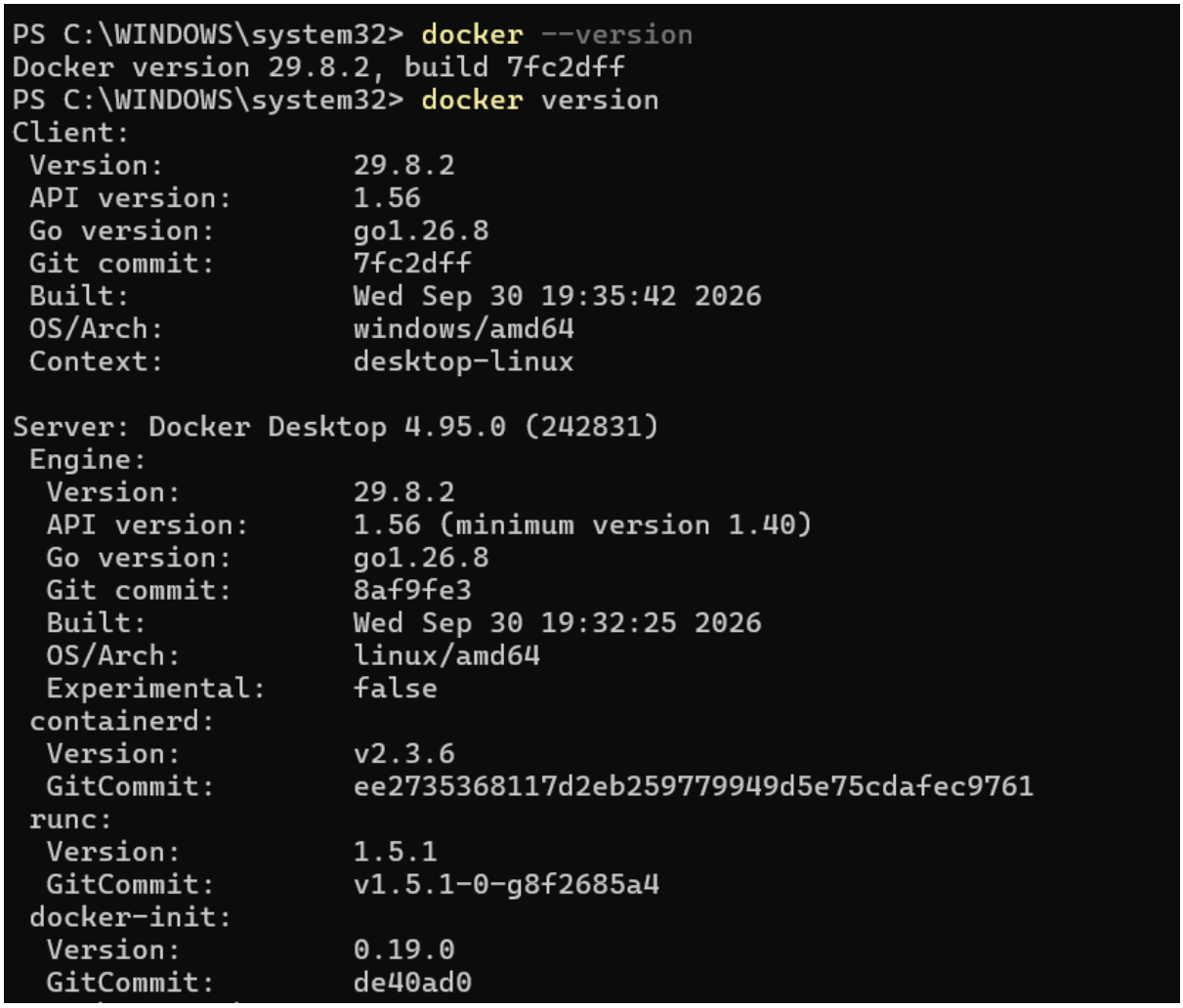
Step 6: Verify Docker Client and Server

Executed:

docker version

This command displays details about the Docker client and server. The client handles command-line requests, while the server (Docker Engine) manages images, containers, and other Docker resources.

Screenshot 6:



Step 7: Run the Hello World Container

Executed:

docker run hello-world

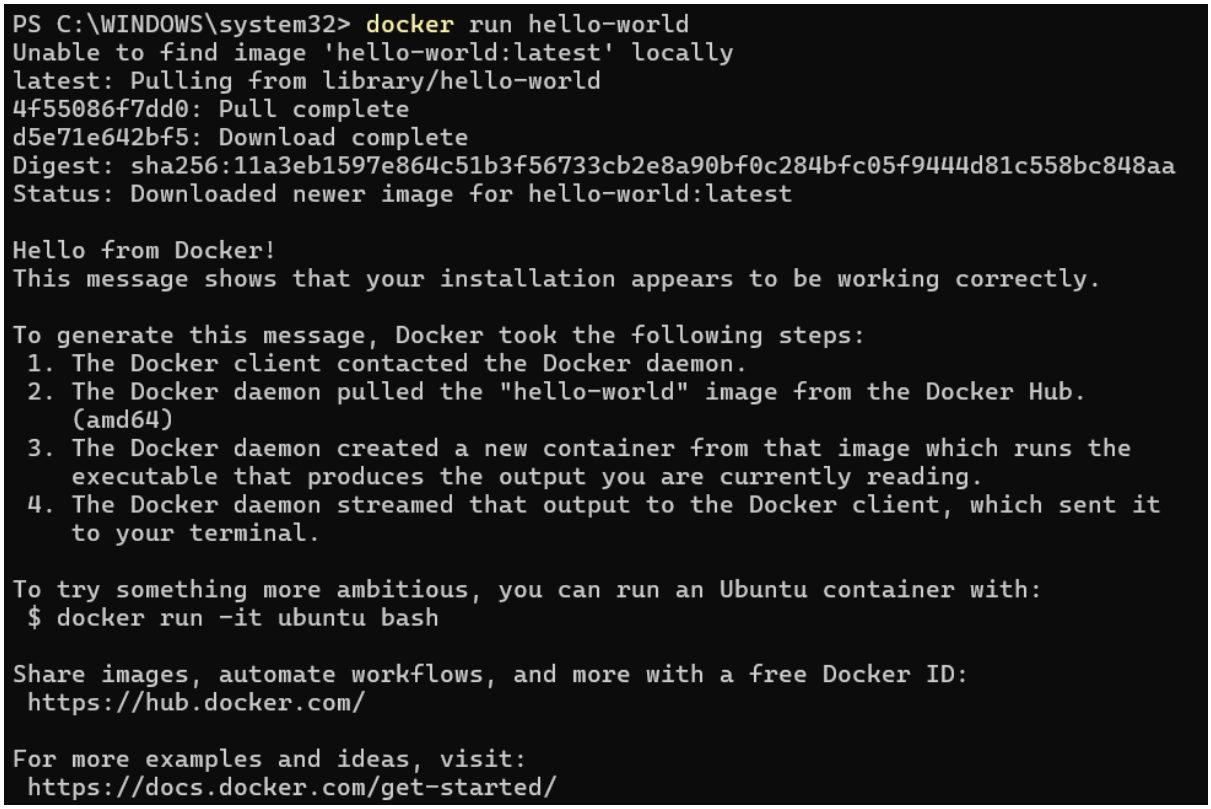
Docker downloaded the hello-world image if it was not already available, created a container from the image, and executed it.

The expected success message begins with:

Hello from Docker!

This confirms that Docker can download an image and run a container.

Screenshot 7:



Step 8: Verify the Container

Executed:

docker ps -a

Actual output:

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

59fb5aa8e133 hello-world "/hello" 45 seconds ago Exited (0) 44 seconds ago vibrant_tu

Explanation:

- docker ps -a lists all containers, including stopped containers.
- CONTAINER ID identifies the container.
- IMAGE shows the image used to create it.
- COMMAND shows the command executed inside the container.
- STATUS shows whether the container is running or has exited.
- Exited (0) confirms that the container completed successfully.

Screenshot 8:



5. Validation Steps

Validation	Command or method	Expected result	Status
Windows version	winver	Windows 11	Verified
WSL installation	wsl --status	Default version 2	Verified
Ubuntu setup	wsl --list --verbose	Ubuntu distribution listed	Verify saved output
Docker Desktop	Docker Desktop dashboard	Engine running	Passed
Docker CLI	docker --version	Version information	Verify saved output
Docker Engine	docker version	Client and server details	Verify saved output
Container execution	docker run hello-world	Success message	Passed, based on subsequent container result
Container listing	docker ps -a	hello-world container listed	Passed

6. Explanation

Docker Desktop provides a local environment for building, running, and managing containers on Windows. WSL 2 provides the Linux environment and kernel integration used by Docker Desktop's recommended WSL backend. The Docker CLI allows users to interact with the Docker Engine through commands.

In this task, the hello-world image was used to validate the installation. Docker created a container from that image, executed its program, and exited successfully. The docker ps -a command confirmed the container's existence and showed Exited (0), demonstrating that the test completed normally.

7. Real-Time Use Case

Docker is widely used in DevOps to package applications and their dependencies into portable containers. For example, a developer can build a web application image, test it locally, and then deploy the same image to a testing or production environment.

Docker can be integrated with CI/CD tools such as Jenkins to automate image builds, testing, tagging, and pushing images to a container registry. This helps maintain consistency between development, testing, and production environments.

8. Common Errors and Troubleshooting

Error or issue	Possible cause	Solution
docker is not recognized	Docker CLI unavailable or PATH not updated	Confirm Docker Desktop is installed and running; open a new PowerShell window.
WSL is not installed	Required Windows components missing	Run wsl --install in administrator PowerShell.

WSL 2 cannot start	Virtual Machine Platform or hardware virtualization unavailable	Enable required Windows features, restart, and verify firmware virtualization.
Docker Engine fails to start	WSL or Docker Desktop startup problem	Check WSL status and restart Docker Desktop.
Cannot connect to Docker Engine	Engine not running or Docker context misconfigured	Wait for Docker Desktop to finish starting, then run docker version.
Image download fails	Internet, proxy, or registry connectivity issue	Verify internet connectivity and Docker registry access.
Container shows Exited (0)	Short-lived container finished successfully	This is expected for hello-world; use a long-running application container for a running-state test.

9. Security and Resource Considerations

- Use trusted container images and keep Docker Desktop updated.
- Avoid running unknown containers with elevated privileges.
- Review container port mappings before exposing services.
- Monitor Docker Desktop's memory and disk usage.
- Remove unused containers and images when they are no longer needed.
- Remember that docker ps shows running containers, while docker ps -a includes stopped containers.

10. Conclusion

Task 3 completed successfully.

Docker Desktop is running on Windows 11, WSL 2 has been configured, and the hello-world container completed successfully with exit code 0. The docker ps -a output confirms that the container was created and executed.